МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки»

Лабораторная работа №1
по курсу «Программирование графических процессоров»

Освоение программного обеспечения для работы с технологией CUDA. Примитивные операции над векторами. Сложение векторов (вариант 1)

Выполнил: Голев Андрей Михайлович
Группа: М8О-401Б-23
Преподаватель: А.Ю. Морозов

Москва, 2026

Условие

Цель работы — познакомиться с технологией CUDA: освоить инструменты разработки и написать свою первую программу на ней, реализующую одну из базовых операций над векторами.

Мой вариант — №3, поэлементное умножение векторов. Во входном файле на первой строке задано n — размер векторов, а на следующих двух строках — по n вещественных чисел: элементы первого и второго векторов. Требуется вывести n чисел — произведения элементов с одинаковыми индексами. Вещественный тип данных — double, ответ выводится с относительной точностью 10^{-10} (я выбрал экспоненциальный формат с десятью знаками после запятой). Ограничение на размер: $n < 2^{25}$. Например, при n = 3 и векторах «1 2 3» и «4 5 6» правильный ответ — «4.0000000000e+00 1.0000000000e+01 1.8000000000e+00».

Программное и аппаратное обеспечение

Писал и отлаживал код на своём ноутбуке MacBook Pro (Apple M3 Pro, 36 ГБ памяти, macOS 15.7). Так как CUDA на macOS не поддерживается, сборку, запуски и все замеры времени я выполнял в Google Colab на графическом процессоре Tesla T4; его характеристики перечислены ниже.

Графический процессор — NVIDIA Tesla T4: compute capability 7.5, 40 мультипроцессоров, глобальная память 15 637 086 208 байт (примерно 14,6 ГБ), разделяемая память на блок 48 КБ, константная память 64 КБ, 65 536 регистров на блок, размер варпа 32 нити, максимум 1024 нити на блок, максимальная сетка (2 147 483 647, 65 535, 65 535).

Из программного обеспечения: Ubuntu 24.04.4 LTS (64 бита), компилятор nvcc 13.0 (g++ 13.3.0). Редактор — VS Code, сборка из командной строки. Проверяющая система компилирует программы nvcc 12.0 на Ubuntu 24.04 — мой код собирается там без каких-либо изменений.

Метод решения

Работа программы устроена следующим образом. Из стандартного ввода считываются n и элементы двух векторов, они размещаются в массивах double на хосте (malloc); под результат отводится третий массив такого же размера. На устройстве выделяется память под все три массива (cudaMalloc), исходные данные копируются на GPU (cudaMemcpy), после чего запускается ядро, вычисляющее c[i] = a[i] · b[i] для каждой пары элементов. Готовый результат копируется обратно на хост и печатается в формате %.10e.

По условиям работы количество вычислительных потоков должно быть фиксированным и не зависеть от размера входных данных, поэтому в ядре применён grid-stride цикл: каждая нить с глобальным индексом g обрабатывает

элементы g, g + stride, g + 2·stride и так далее, где stride — суммарное число запущенных нитей. Сетка при этом всегда одна и та же, каким бы ни было n.
Программа не должна аварийно завершаться ни при каких условиях, поэтому проверяется результат каждого вызова CUDA API и всего ввода-вывода. Если что-то не удалось — не выделилась память, не запустилось ядро, не разобрались входные данные — в stderr выводится строка «ERROR: …», и программа заканчивает работу с кодом возврата 0. При написании я опирался на официальную документацию CUDA C Programming Guide (docs.nvidia.com).

Описание программы

Программа целиком умещается в один файл lab1.cu и состоит из ядра и функции main.

vecMulKernel(const double *a, const double *b, double *c, long long n) — ядро: каждый поток проходит по закреплённым за ним элементам grid-stride циклом и перемножает соответствующие элементы векторов a и b.
main() — вся остальная логика: разбор входа (scanf), выделение памяти на хосте и устройстве (malloc/cudaMalloc), копирование туда и обратно (cudaMemcpy), запуск ядра, печать ответа (printf, формат %.10e) и освобождение ресурсов; каждый шаг проверяется на ошибки.

Использованные типы: double — элементы векторов, long long — размер и индексы, cudaError_t — коды ошибок CUDA. Рабочая конфигурация сетки — <<<256, 256>>>, то есть 65 536 нитей; она не меняется в зависимости от n. Все прочие конфигурации из таблицы ниже запускались только в рамках замеров.

Результаты

Время работы ядра я измерял событиями CUDA (cudaEventRecord / cudaEventSynchronize): событие start записывается перед запуском ядра, событие stop — сразу после, затем берётся время между ними. Для каждой конфигурации выполнялось 3 прогревочных запуска и 10 замеряемых, в таблице — среднее по 10 запускам. Размеры входа взял три: маленький $n = 10^3$, средний $n = 10^6$ и почти предельный $n = 2^{25} - 1 \approx 3{,}36 \cdot 10^7$. Для сравнения я написал CPU-версию без CUDA (g++ -O2, один поток) и замерял в ней время самого цикла умножения.

Конфигурация (блоки × нити)	Всего нитей	n = 10^3, мкс	n = 10^6, мс	n = $2^{25} - 1$, мс
<<<1, 32>>>	32	24,1	16,11	390,34
<<<1, 64>>>	64	17,7	5,78	195,29
<<<1, 128>>>	128	12,5	2,93	99,34
<<<1, 256>>>	256	10,9	1,53	50,37
<<<1, 512>>>	512	8,7	0,82	26,67
<<<1, 1024>>>	1 024	9,5	0,48	15,52
<<<32, 32>>>	1 024	7,6	0,40	13,61
<<<1024, 1>>>	1 024	10,6	0,79	26,90
<<<32, 256>>>	8 192	8,6	0,10	3,44
<<<128, 64>>>	8 192	8,1	0,11	3,55
<<<256, 256>>>	65 536	10,0	0,10	3,29
<<<1024, 256>>>	262 144	9,6	0,10	3,28
<<<1024, 512>>>	524 288	12,6	0,10	3,29
<<<1024, 1024>>>	1 048 576	20,8	0,10	3,28
CPU (g++ -O2)	1 поток	0,7	1,45	74,00

Из таблицы видно, что ядро упирается в пропускную способность памяти. На предельном входе читаются два вектора и записывается один — всего 24·n байт, то есть около 805 МБ; лучшее время 3,28 мс даёт примерно 246 ГБ/с, порядка 77 % от теоретического максимума T4 (около 320 ГБ/с). Пока нитей мало, время почти строго обратно пропорционально их количеству: 32 нити — 390,34 мс, 64 — 195,29 мс, 128 — 99,34 мс, 256 — 50,37 мс, 512 — 26,67 мс, 1024 — 15,52 мс; после ~8192 нитей время выходит на плато примерно 3,3 мс. При равном числе нитей конфигурация <<<1024, 1>>> заметно уступает <<<32, 32>>> (26,90 мс против 13,61 мс на предельном входе): блок из одной нити задействует лишь одну из 32 дорожек варпа. На маленьком входе (n = 10^3) всё решают накладные расходы на запуск ядра (7,6–24,1 мкс), поэтому от конфигурации почти ничего не зависит, и CPU оказывается быстрее (0,7 мкс против 7,6 мкс у лучшей конфигурации). На среднем входе лучшая конфигурация обгоняет CPU примерно в 14 раз (0,10 мс против 1,45 мс), на предельном — примерно в 22 раза (3,28 мс против 74,00 мс). Рабочая конфигурация <<<256, 256>>> практически не отличается от лучшей: 3,29 мс против 3,28 мс на предельном входе и столько же 0,10 мс на среднем.

Выводы

Реализованная операция относится к классу поэлементных (map) операций над векторами: каждый элемент результата вычисляется независимо от остальных. Типичные задачи такого рода — масштабирование массивов и сигналов, поэлементное умножение на маску или весовые коэффициенты, подготовка данных перед редукциями (например, вычисление скалярного произведения через

умножение с последующим суммированием), а также простые преобразования изображений.

Полученные результаты объясняются тем, что ядро ограничено пропускной способностью памяти: производительность растёт с числом нитей лишь до насыщения памяти (примерно 8192 нитей), а дальше выходит на плато. На больших объёмах данных GPU-версия быстрее однопоточной CPU-версии примерно в 22 раза; на малых объёмах GPU проигрывает из-за накладных расходов на запуск ядра.